

SIH26171

On-device Visual Perception for Light-weight Browser Agents

Indian Space Research Organisation (ISRO) | Theme: Smart Automation | Category: Software
Deadline: 20 September 2026

1 Problem Understanding and Goal

- **Context:** AI agents assist users by understanding screen states, but most pipelines run server-side and require sending sensitive data.
- **Goal:** A *privacy-preserving vision agent that runs in the browser*: local vision model on-device, only sanitized/redacted data sent to a server LLM/VLM, which returns actionable commands.
- **Key idea:** Balance server reasoning power with strict client-side data privacy.

2 Client-Side Components (Browser Extension / JS)

1. Screen Capture

- Manifest v3 Chrome/Firefox extension; capture tab via `chrome.tabs.captureVisibleTab()` or `getDisplayMedia()`.

2. Local Vision Processing

- Lightweight model: **MobileViT** / **Tiny-ViT** run via **Transformers.js** / **ONNX Runtime Web**.
- Accelerated by **WebGPU** / **WebAssembly**.
- Produces structured screen representation: UI element bounding boxes, text regions, icons.

3. Privacy-Preserving Filter

- DOM-based analysis to find PII elements (password fields, inputs labelled name/email/phone/SSN/address).
- Visual redaction: **masking** / **bounding-box blackout** over sensitive regions.
- Face detection via **BlazeFace** (WebGPU) → **blur** faces.
- Result: sanitized logo/frame with PII visually redacted before any network call.

3 Server-Side Components

1. Transmission

- Send only the *anonymized* visual context (redacted image / structured DOM) to the server; no raw sensitive pixels ever leave the client.

2. LLM/VLM Interpretation

- Server runs a VLM aware of the redaction scheme (e.g., LLaVA, CogVLM, GPT-4V / Claude Vision).
- Returns a structured action: e.g. "click submit", "scroll down".

3. Action Execution (loop back to client)

- Extension executes action via DOM (querySelector().click()), then re-captures.
- End-to-end agent loop: capture → sanitize → send → interpret → execute.

4 Evaluation Metrics (Per Problem Statement)

Metric	Weight
Accuracy of visual context from screen	25%
Recall & precision for detection of sensitive/PII data	20%
Precision of redaction	20%
Client-side resource utilization	20%
Overall end-to-end latency of the task	15%

5 Work Plan / Milestones

Phase	Activities	Timeline
1. Extension	Manifest v3 scaffold + screen capture	Week 1–2
2. Local Model	Tiny-ViT on WebGPU/ONNX, screen parsing	Week 2–4
3. Redaction	PII detection + masking + face blur	Week 4–5
4. Server	VLM endpoint returning actions	Week 5–6
5. Agent Loop	End-to-end task execution	Week 6–7
6. Optimization	Latency & resource tuning + metrics	Week 7–9

6 Technology Stack

JavaScript/TypeScript, WebExtensions API, WebGPU, WebAssembly, ONNX Runtime Web, Transformers.js, BlazeFace, Node.js server, LLaVA/CogVLM.

Key Note

Weights show that a well-engineered **privacy filter** (redaction precision + PII detection = 40%) matters nearly as much as the vision accuracy. A clean, demonstrable redaction pipeline is the key differentiator.

7 Deliverables

- Browser extension (client-side) + server code.
- Live prototype demonstrating an end-to-end assistance task.
- Metrics reporting (accuracy, PII recall/precision, redaction precision, resource use, latency).

- Source code, README, demo video, architecture/presentation docs.